Leant

a Djex-based synthesis REPL for Lean 4

an overview and tutorial, with a detailed tour of `:synth`

An **experimental** tool in active development. Interfaces, output, and capabilities change frequently.

github.com/VladimirReshetnikov/Leant August 2026.

Contents

1 Introduction

Leant is an interactive read–eval–print loop for Lean 4, in the spirit of GHCi: you type expressions and they are evaluated, you type declarations and they enter the session, and a family of colon-commands gives you type queries, documentation, search, interactive proving, and automatic *term synthesis* with machine-checked answers. The synthesis command takes up half of this document.

Lean 4 itself is normally driven from an editor: the language server shows goals and diagnostics as you edit a file. Leant complements that workflow with a conversational one. It is aimed at exploration (trying a lemma, poking at a definition, asking *is this type even inhabited?*), where the unit of work is a line, not a file.

experimental

Leant is an **experimental project in active development**. It is a research vehicle, not a supported product: commands appear, change shape, and disappear between commits; output formats are not stable; and some features are bounded approximations by design. Every transcript in this document was produced by the tool at the date on the title page and may not match the tool you build tomorrow. When this document and `:help` disagree, believe `:help`.

Leant is implemented in Haskell and speaks the backend protocol directly; term synthesis (§5) embeds the Djex synthesis library in-process.

Under the hood, Leant drives the community `leanprover-community/repl` binary over a JSON protocol: every input becomes a command against a persistent Lean environment, and every claim Leant makes, including every synthesized term, is checked by that backend before it is shown. Sessions survive backend crashes (the session history replays automatically), can be saved and restored (`:pickle`/`:unpickle`, including synthesis-ready companions for Leant-created snapshots), and can be recorded as transcripts.

2 Getting started

Build the binary once with `cabal build exe:leant` (GHC 9.12.4; the vendored `lib/Djex` submodule must be initialized), then run `leant.cmd`, or the built executable directly. Leant finds the Lean backend the same way LeanInteract's cache laid it out, or accepts an explicit `--repl-exe`/`LEANT_BACKEND`. Without a Lake project it runs against the plain Lean 4 standard library, which is also the least memorable thing about it: no imports, subsecond startup.

A session begins as a calculator and escalates quietly:

session

```
λ> 2 + 2
4
λ> it * 10
40
λ> def double (n : Nat) : Nat := n + n
λ> double 21
42
```

```
λ> :type double
double : Nat → Nat
```

Expressions are tried with `#eval` and fall back to `#check`; the last evaluated value is bound as `it`, GHCi-style. Declarations (`def`, `theorem`, `inductive`, …) extend the session environment, and any `#`-command (`#eval`, `#check`, `#print axioms`) passes straight through. Multi-line input opens automatically when a line is syntactically incomplete (type a `structure` header and just keep going), and an empty line submits it.

3 The commands

3.1 Session and evaluation

`:help, :h, :?`	show the command summary
`:quit, :q`	exit
`:type EXPR, :t`	show the type of `EXPR` (`#check`)
`:info NAME, :i`	show the definition of `NAME` (`#print`)
`:load FILE, :l`	reset the session and load a `.lean` file
`:reload, :r`	reload the last loaded file
`:import MOD`	add an import (rebuilds the session; unavailable over an opaque snapshot)
`:imports`	list active imports, or imports restored by `:reset`
`:undo`	revert the last state-changing command
`:reset`	clear definitions or a snapshot base (keeps configured imports)
`:history`	show commands after the current import/snapshot base
`:set OPT VAL`	a Lean `set_option` that persists in the session

3.2 Discovery

`:browse [NS]`	list declarations in a namespace, or the session's own
`:browse! NS`	…including compiler-generated auxiliaries
`:doc NAME`	show a docstring
`:search TEXT`	search declaration names, case-insensitively
`:search? TYPE`	proof search: what proves `TYPE`? (via `exact?`)

3.3 Proving and synthesis

`:prove [PROP]`	prove `PROP` interactively, tactic by tactic (without an argument: resume the last `sorry`)
`:synth TYPE`	synthesize verified terms of `TYPE`; candidates are bound as `it1` (= `it`), `it2`, …(§5)
`:set synth-engine E`	`djinn` (default) | `exference` | `both`
`:set synth-steps N`	Exference's step budget (default 4096)
`:set synth-classical B`	offer classical candidates for constructively refuted goals (`on`|`off`, default on)

`:set synth-library B`	library premises for recursive inductives (`on`|`off`, default on; §5.6)

3.4 Persistence and utility

`:transcript [F|on|off]`	record a transcript of the session
`:timestamps [on|off]`	timestamp transcript entries
`:pickle FILE`	save the environment and synthesis companion
`:unpickle FILE`	restore an environment as a new undo/history base
`:env`	show the backend environment id
`:time`	toggle per-command timing
`:! CMD`	run a shell command

4 Interactive proving

`:prove` turns the prompt into a tactic loop against the backend's proof-state protocol. Every line is a tactic; `:undo` pops one step; `:qed NAME` assembles the accumulated script into a theorem and replays it into the session, so the result is usable afterwards. Each displayed proof state is followed by an automatically discovered tactic suggestion that Lean has already checked. The candidate probes are shaped by the goal and its hypotheses: an `intro` suggestion names the binders it would introduce, a disjunction goal is probed with `left`/`right`, a hypothesis whose type a single step can take apart is probed with `cases` or `obtain`, and a data-typed variable the goal mentions is probed with `induction`. The search prefers a candidate that closes the goal outright and annotates the suggestion accordingly (`closes the goal`, `splits into 2 goals`); when no single tactic closes it, a second phase chains quick finishers onto the best progressing candidates (`constructor <;> omega`, or an `obtain` step chained with the `exact` it enables), so even the opening suggestion is often a complete checked proof. The chains also try `simp_all`, unfolding the definitions the goal mentions and calling in `omega` on whatever arithmetic remains, which is how an induction suggestion can arrive as a finished proof: `induction l <;> simp_all [myLen]`, or `induction n <;> simp_all [double] <;> omega`. The suggestion is advisory—it does not advance the state or enter the script—and `:suggest` reprints it without rerunning the search. Below, the very first suggestion is already a complete proof; the session ignores it and proceeds step by step instead. Proof-state identifiers are local to one backend process; if it stops, Leant leaves prove mode and prints the accumulated script rather than submitting a stale identifier after session reconstruction.

session

```
λ> :prove ∀ p q : Prop, (p → q) → p → q ∧ p
entering prove mode — type tactics; :help for commands
⊢ ∀ (p q : Prop), (p → q) → p → q ∧ p
suggestion: exact fun p q a a_1 => ⟨a a_1, a_1⟩  (closes the goal)
⊢> intro p q h hp
p q : Prop
h : p → q
hp : p
⊢ q ∧ p
```

```
suggestion: exact ⟨h hp, hp⟩  (closes the goal)
⊢> exact ⟨h hp, hp⟩
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed mp_and
saved: theorem mp_and : ∀ p q : Prop, (p → q) → p → q ∧ p
```

Accepting a suggestion is just typing it. On a conjunction of two arithmetic facts, the opening suggestion is already the whole chained proof, and the session is two lines long:

session

```
λ> :prove (∀ a b : Nat, a + b = b + a) ∧ (∀ a b : Nat, a + 2 * b = b + b + a)
entering prove mode — type tactics; :help for commands
⊢ (∀ (a b : Nat), a + b = b + a) ∧ ∀ (a b : Nat), a + 2 * b = b + b + a
suggestion: constructor <;> omega  (closes the goal)
⊢> constructor <;> omega
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed two_omegas
saved: theorem two_omegas : (∀ a b : Nat, a + b = b + a) ∧ (∀ a b : Nat, a + 2 * b = b
↪  + b + a)
```

That chain was suggested because *both* halves fall to `omega`. When they need different tools, the suggestion adapts to whichever goal is in focus — arithmetic for the first half, a found library lemma for the second:

session

```
λ> :prove (∀ a b : Nat, a + b = b + a) ∧ (∀ a b : Nat, a * b = b * a)
entering prove mode — type tactics; :help for commands
⊢ (∀ (a b : Nat), a + b = b + a) ∧ ∀ (a b : Nat), a * b = b * a
suggestion: constructor  (splits into 2 goals)
⊢> constructor
— goal 1 of 2 —
case left
⊢ ∀ (a b : Nat), a + b = b + a
— goal 2 of 2 —
case right
⊢ ∀ (a b : Nat), a * b = b * a
suggestion: omega  (closes the goal)
⊢2> omega
case right
⊢ ∀ (a b : Nat), a * b = b * a
suggestion: exact fun a b => Nat.mul_comm a b  (closes the goal)
⊢> exact fun a b => Nat.mul_comm a b
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed add_mul_comm
saved: theorem add_mul_comm : (∀ a b : Nat, a + b = b + a) ∧ (∀ a b : Nat, a * b = b *
↪  a)
```

The probes are shaped by the hypotheses as much as by the goal. An existential hypothesis draws an `obtain` suggestion that names its witness; once the conjunction inside is the thing in the way, the suggested step is the `obtain` that splits it, chained with the `exact` it enables:

session

```
λ> :prove (∃ x : Nat, 2 < x ∧ x < 4) → ∃ y : Nat, 2 < y
entering prove mode — type tactics; :help for commands
⊢ (∃ x, 2 < x ∧ x < 4) → ∃ y, 2 < y
suggestion: intro h
⊢> intro h
h : ∃ x, 2 < x ∧ x < 4
⊢ ∃ y, 2 < y
suggestion: obtain ⟨x, h1⟩ := h
⊢> obtain ⟨x, h1⟩ := h
x : Nat
h1 : 2 < x ∧ x < 4
⊢ ∃ y, 2 < y
suggestion: obtain ⟨h, h2⟩ := h1 <;> exact Exists.intro x h  (closes the goal)
⊢> obtain ⟨h, h2⟩ := h1 <;> exact Exists.intro x h
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed ex_two_lt
saved: theorem ex_two_lt : (∃ x : Nat, 2 < x ∧ x < 4) → ∃ y : Nat, 2 < y
```

A suggestion is not limited to one step. Here the goal needs structural induction on a user-defined function; once `intro` brings the variable into the context, the suggested chain arrives as a complete verified proof:

session

```
λ> def myLen : List Nat → Nat
…>   | [] => 0
…>   | _ :: t => myLen t + 1
…>
λ> :prove ∀ l : List Nat, myLen l = l.length
entering prove mode — type tactics; :help for commands
⊢ ∀ (l : List Nat), myLen l = l.length
suggestion: intro l
⊢> intro l
l : List Nat
⊢ myLen l = l.length
suggestion: induction l <;> simp_all [myLen]  (closes the goal)
⊢> induction l <;> simp_all [myLen]
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed myLen_length
saved: theorem myLen_length : ∀ l : List Nat, myLen l = l.length
```

The same pattern scales to arithmetic recursion. The chain below was assembled from three of the probes at once: `induction` because the goal mentions a data-typed variable, `simp_all` unfolding the definition the goal is about, and `omega` for the arithmetic the simplifier leaves behind — a finished proof of a small theorem about a function defined moments earlier:

session

```
λ> def double : Nat → Nat
…>   | 0 => 0
…>   | n + 1 => double n + 2
…>
λ> :prove ∀ n : Nat, double n = 2 * n
entering prove mode — type tactics; :help for commands
⊢ ∀ (n : Nat), double n = 2 * n
suggestion: intro n
⊢> intro n
n : Nat
⊢ double n = 2 * n
suggestion: induction n <;> simp_all [double] <;> omega  (closes the goal)
⊢> induction n <;> simp_all [double] <;> omega
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed double_two_mul
saved: theorem double_two_mul : ∀ n : Nat, double n = 2 * n
```

A `sorry` in an ordinary declaration prints its goal and offers the same loop via bare `:prove`. Prove mode also cooperates with `:synth`; §5.9 shows what that buys.

5 Term synthesis: `:synth`

`:synth TYPE` answers the question *"write me a term of this type"*; read through propositions-as-types, that is *"prove this."* Sometimes it answers the stronger question *"show me that no such term exists."* It is built on the vendored Djex library, linked in-process. Djex began by merging two classic Haskell program synthesizers behind one checked synthesis foundation: **Djinn**, a complete and terminating proof search for intuitionistic propositional logic (Dyckhoff's LJT calculus) that emits programs, and **Exference**, a ranked heuristic search under explicit budgets. It has since moved well beyond both originals. It fixes a long list of their bugs; the worst of them let a search that had merely approximated a type report that type uninhabited, and an exhausted search over an approximated space now counts for nothing. It implements much better type-class support, with a unified, validated constraint contract shared by both engines: Exference resolves givens, superclasses, and instances, while Djinn checks contexts and synthesizes dictionary-independent terms. And it adds a growing level of support for rank-N and impredicative types: goal-side quantifiers open through polarity-aware plan families, quantified hypotheses are instantiated at a bounded set of sequent-supplied types, and impredicative instantiation is admitted under a guard that never invents a polytype the query did not supply. Djinn's current context-free hypothesis rule reaches four leading binders without raising its fixed search caps; chains with five or more stay opaque. Its singleton, pairwise, triple, and quadruple open/opaque frontiers grow quartically and cover every choice through nine independent quantified sites without enumerating a power set. That scope is still being improved, and Leant keeps the boundary to the engines narrow on purpose, so each improvement arrives by version bump. The engines speak Haskell types; Leant translates Lean goals into their fragment and their answers back into Lean.

Three rules run through the design:

- **The engine is never trusted.** Every candidate is re-elaborated by the Lean backend against the exact goal (`example : (T) := term`) before you see it. A rendering bug or an engine bug costs a dropped candidate, never a wrong answer.
- **Refusals come with reasons.** A goal outside the supported fragment is turned away with a note saying what fell outside, not quietly mangled into something answerable.
- **Negative verdicts are labeled by strength.** "Provably uninhabited" appears only when the translation was complete and lossless; anything weaker is reported as "no term found within bounds," which claims nothing.

Transcripts in this section are lightly abridged: trailing candidates beyond the ones shown, and trailing truncation notes, are elided. Every line that does appear is a verbatim record of a live session.

5.1 Higher-order plumbing

The sweet spot is the structural fragment: arrows, products, sums, ⊥, ⊤, negation, `Iff`, and quantifiers over opaque variables. These are the "plumbing" terms one writes constantly. Free capital identifiers are auto-bound, so quick queries stay quick:

session

```
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1  fun f g x => f x (g x)
λ> :synth (A × (B ⊕ C) → (A × B) ⊕ (A × C))
  it1  fun ⟨x, y⟩ => match y with | .inl z => .inl ⟨x, z⟩ | .inr w => .inr ⟨x, w⟩
λ> :synth (∀ p q : Prop, (p ↔ q) → ¬p → ¬q)
  it1  fun _ _ ⟨_, f⟩ k x => k (f x)
```

The first is the S combinator; the second distributes a product over a sum; the third is contraposition across an `Iff`, with the backward implication projected out by the pattern `⟨_, f⟩`. Binders are named by role (functions `f g h`, values `x y z`, negations and continuations `k`), a small thing that makes large candidates much easier to read.

The everyday combinators all come out on the first try, and each first candidate is the term you would have written yourself — `flip`, composition, `curry`, `uncurry`, and product associativity:

session

```
λ> :synth ((a → b → c) → b → a → c)
  it1  fun f x y => f y x
λ> :synth ((b → c) → (a → b) → a → c)
  it1  fun f g x => f (g x)
λ> :synth ((A × B → C) → A → B → C)
  it1  fun f x y => f ⟨x, y⟩
λ> :synth ((A → B → C) → A × B → C)
  it1  fun f ⟨x, y⟩ => f x y
λ> :synth (((A × B) × C) → A × (B × C))
  it1  fun ⟨⟨x, y⟩, z⟩ => ⟨x, ⟨y, z⟩⟩
```

Candidates are ranked smallest-first and *bound into the session* as `it1`, `it2`, …, with bare `it` the best one. They are ordinary definitions; you can evaluate them immediately:

session

```
λ> :synth (a → a → a)
  it1  fun _ x => x
  it2  fun x _ => x
λ> #eval it2 "left" "right"
"left"
```

Both inhabitants of $a \to a \to a$ that matter, and picking one is just using its name. For programs, as opposed to proof-irrelevant propositions, the choice between candidates is the whole point.

An `Iff` goal is treated as a pair of implications, so both directions of a logical identity arrive in one anonymous constructor. De Morgan and the absorption law:

session

```
λ> :synth (∀ p q : Prop, ¬(p ∨ q) ↔ ¬p ∧ ¬q)
  it1  fun _ _ => ⟨fun k => ⟨fun x => k (.inl x), fun y => k (.inr y)⟩, fun ⟨k1, k2⟩ z
  ↪  => match z with | .inl w => k1 w | .inr x1 => k2 x1⟩
λ> :synth (∀ p q : Prop, p ∧ (p ∨ q) ↔ p)
  it1  fun _ _ => ⟨fun ⟨x, _⟩ => x, fun y => ⟨y, .inl y⟩⟩
```

Under propositions-as-types, the proof terms *are* plumbing, and the synthesized spellings make that visible: `Iff` symmetry is a swap, and `¬(p ∧ q) ↔ (p → ¬q)` is nothing but currying:

session

```
λ> :synth (∀ p q : Prop, (p ↔ q) → (q ↔ p))
  it1  fun _ _ ⟨f, g⟩ => ⟨g, f⟩
λ> :synth (∀ p q : Prop, ¬(p ∧ q) ↔ (p → ¬q))
  it1  fun _ _ => ⟨fun k x y => k ⟨x, y⟩, fun k1 ⟨z, w⟩ => k1 z w⟩
```

The textbook curiosities are in range too. No proposition is equivalent to its own negation, and the synthesized proof is the classic self-application trick — the term below applies `k` to itself through `f` twice, the propositional shadow of $\omega\,\omega$:

session

```
λ> :synth (∀ p : Prop, (p ↔ ¬p) → False)
  it1  fun _ ⟨k, f⟩ => k (f (fun x => k x x)) (f (fun y => k y y))
```

5.2 Programs you already know

Some types have one sensible inhabitant, and asking for it by type is quicker than remembering which library corner it lives in. The bind of the reader monad, then the bind of the state monad:

session

```
λ> :synth ((S → A) → (A → S → B) → S → B)
  it1  fun f g x => g (f x) x
λ> :synth ((S → A × S) → (A → S → B × S) → S → B × S)
  it1  fun f g x => match f x with | ⟨a, b⟩ => g a b
  it2  fun f g x => match f x with | ⟨a, _⟩ => g a x
  it3  fun f g x => match f x with | ⟨a, b⟩ => (match g a x with | ⟨c, _⟩ => ⟨c, b⟩)
```

The first candidate threads the state correctly. Now look at the second: it is type-correct and runs `g` on the *initial* state, throwing away whatever `f` did to it. That is the classic state-threading bug, and the type admits it just as happily. Types alone cannot tell these apart, which is why all candidates are shown and each is one keystroke from a test run. Continuation-passing style works too; this is the bind of the continuation monad:

session

```
λ> :synth ((A → R) → R) → (A → (B → R) → R) → (B → R) → R
  it1  fun f g h => f (fun x => g x h)
```

With the inductive expansion of §5.5 the same game extends to data. Here is `Option.bind`, with the lazy `.none` candidate ranked first and the real one second. When in doubt, run one:

session

```
λ> :synth (∀ a b : Type, Option a → (a → Option b) → Option b)
  it1  fun _ _ _ _ => .none
  it2  fun _ _ x f => match x with | .none => (.none) | .some y => f y
```

5.3 Rank-N and impredicative goals

A polymorphic hypothesis is not just cargo: Djex instantiates it at types the goal itself supplies — including, under a guard, at *polymorphic* ones. Here the first candidate applies the identity hypothesis to the whole goal `Q → Q`, an impredicative instantiation:

session

```
λ> :synth ((∀ p : Prop, p → p) → Q → Q)
  it1  fun f => f _
  it2  fun _ x => x
  it3  fun f x => f _ x
```

And a Church-encoded pair converts into a real conjunction — the quantified hypothesis is instantiated once at `p` and once at `q`, fed the matching projection each time:

session

```
λ> :synth (∀ p q : Prop, (∀ r : Prop, (p → q → r) → r) → p ∧ q)
  it1  fun _ _ f => ⟨f _ (fun x _ => x), f _ (fun _ y => y)⟩
```

(Trailing candidates are elided.) Explicit `∀` binders — leading, nested, trailing, or interleaved — are woven into the candidate's lambda automatically, and uses of quantified hypotheses

get placeholder type arguments wherever Lean needs them (`f _ x`), so bounded rank-N candidates verify. Full impredicative inhabitation is undecidable; beyond the guard the answer is "no term found within bounds" and nothing stronger.

Context-free instantiation chains extend to four leading binders. Leant inserts the inferred type arguments which Djinn leaves implicit, then asks Lean to verify every rendered candidate against the original type:

session

```
λ> :synth (∀ A B C D R : Type, (∀ a b c d : Type, a → b → c → d → R) → A → B → C → D →
↪  R)
  it1  fun _ _ _ _ _ f => f _ _ _ _
  …
  it5  fun _ _ _ _ _ f x y z w => f _ _ _ _ x y z w
```

Vacuous local providers can retain the same query-supplied choice even when their result is an opaque Lean type. Here `Demo.Token` becomes a fixed ambient query rigid, not an unresolved inference variable:

session

```
λ> axiom Demo.Token : Type
λ> :synth ((∀ x : Type, x → x) → ({a : Type 1} → Demo.Token) → Demo.Token)
  it1  fun _ x => @x (∀ (a0_0 : Type _), a0_0 → a0_0)
λ> :set synth-engine exference
synth engine: exference
λ> :synth ((∀ x : Type, x → x) → ({a : Type 1} → Demo.Token) → Demo.Token)
  it1  fun f x => f _ (@x (∀ (a0_0 : Type _), a0_0 → a0_0))
```

This visible specialization is available only for a fully vacuous, context-free leading provider chain, and it selects only a closed proper type already present in the checked query. Fixed ambient rigids may occur in the provider result; a free flexible variable rejects the route. A local value has no global binder name, so Leant renders a positional `@` application. For a quantified argument it tries binder domains `_`, `Type _`, and `Prop`, in that order, and backend verification keeps the first spelling which elaborates. Named globals retain source-directed applications. Each domain lane retains at most twelve site-and-style spellings, for a hard 36-variant bound only when the local quantified argument is domain-sensitive; ordinary duplicate spellings collapse to the historical smaller group.

An active Lean instance head can establish a provider-local choice even when the polytype does not occur in the query. In this session the goal contains only `Gap.Token`; the instance for `Gap.C` determines the quantified argument of the exact provider:

session

```
λ> axiom Gap.Token : Type
λ> class Gap.C (a : Type 1) : Prop where witness : True
λ> instance : Gap.C (∀ x : Type, x → x) := ⟨True.intro⟩
λ> axiom Gap.polyGlobal {a : Type 1} [Gap.C a] : Gap.Token
λ> :set synth-engine djinn
synth engine: djinn
λ> :synth Gap.Token
  it1  Gap.polyGlobal («a» := (∀ (a0_0 : _), a0_0 → a0_0))
```

```
λ> :set synth-engine exference
synth engine: exference
λ> :synth Gap.Token
  it1  Gap.polyGlobal («a» := (∀ (a0_0 : _), a0_0 → a0_0))
λ> :set synth-engine both
synth engine: both
λ> :synth Gap.Token
  it1  Gap.polyGlobal («a» := (∀ (a0_0 : _), a0_0 → a0_0))
```

An active head can also determine a higher-kinded binder which is absent from the provider body. Leant preserves the argument's bounded `Type -> … -> Type` kind, so both checked engines keep this constraint-only application visible:

session

```
λ> namespace Higher
λ> axiom Wrap : Type → Type
λ> class VacuousChoice (F : Type → Type) : Prop where witness : True
λ> instance : VacuousChoice Wrap := ⟨True.intro⟩
λ> axiom VacuousToken : Type
λ> axiom vacuous {F : Type → Type} [VacuousChoice F] : VacuousToken
λ> end Higher
λ> :set synth-engine djinn
synth engine: djinn
λ> :synth Higher.VacuousToken
  it1  Higher.vacuous («F» := Higher.Wrap)
λ> :set synth-engine exference
synth engine: exference
λ> :synth Higher.VacuousToken
  it1  Higher.vacuous («F» := Higher.Wrap)
```

One exact provider may retain several distinct successful instance-head vectors. In the same transcript, heads for `AlternativeChoice Wrap` and `AlternativeChoice (Pair Nat)` produce exactly `Higher.alternative («F» := Higher.Wrap)` and `Higher.alternative («F» := (@Higher.Pair Nat))`. Standalone Djinn ranks `Wrap` first, standalone Exference ranks `Pair Nat` first, and combined mode uses Djinn's order after stable exact-spelling deduplication. That is an engine-ranking difference: every mode must retain both exact alternatives once. The transcript also carries one heterogeneous two-binder vector whose positional kind arities are one and two, and requires `Higher.multiVacuous («F» := Higher.Wrap) («G» := (@Higher.Triple Nat))` in all three modes.

Discovery opens no more than four leading type binders on one exact provider while retaining its erased instance constraints. It inspects at most 32 active heads per provider in Lean resolver order. Each selected head and its complete constraint closure run under restored metavariable state. The seed head stays fixed while its returned instance subgoals and every other provider constraint are solved in that same context; a failure or unresolved binder rejects the head as a whole. One success contributes one complete vector in leading-binder order, never an uncorrelated scalar pool. For each argument, discovery also reduces its type to the admitted first-order kind language and retains the number of `Type` arrow domains. Zero denotes proper type; positive counts preserve bare and partially applied constructors.

Leant retains at most 16 alpha-distinct vectors per provider and passes no more than 32 provider/vector associations to Djex. Each vector has exact provider arity and at most four arguments. The command takes that aggregate prefix before an argument affects family planning, rigidity, or translation. Live provider metadata uses `(instantiations (args (kinded N ...) ...))`. Leant folds each bounded arrow count into a Djex `GroundKind`, pairs it with the translated type, and calls `runDjinnQueryWithKindedInstantiationAssignments` or `runExferenceQueryWithKindedInstantiationAssignments`. Live discovery, wire parsing, and engine filtering all admit at most 64 `Type`-arrow domains, whose simple right-associated kind has `2 * 64 + 1 = 129` constructors. After an assignment passes Djex's provider, scheme, context, and exact-arity checks, the pinned adapters productively preflight all of that assignment's supplied kinds before recursive kind inference, same-provider comparison, kind conversion, or paired type elaboration. Oversized and cyclic caller-built kinds therefore fail finitely. Metadata-free and binder-only inventories remain valid. Historical exact arguments stay in their original vectors and default each position to proper kind; the `(candidates ...)` form parses as proper-kind unary vectors only. A vector containing depth truncation (`FDepth`) or an instance binder (`FInst`) is discarded in full, including an instance binder revealed by reducing an alias. Provider-prefix fallback keeps metadata attached to its exact declaration. The checked runners validate provider identity, arity, supplied positional kinds, closure, and context before consuming each vector once without Cartesian reconstruction. Thus a later or alpha-identically typed provider cannot donate its assignment to an earlier one.

Leant also retains applications whose arguments are proper types. A bound first-order constructor remains a higher-kinded variable in Djex; an opaque Lean family receives one rigid nominal head across its occurrences. Both engines can therefore see a query-supplied polytype without learning any hidden implementation:

session

```
λ> axiom Wrap : Type 1 → Type
λ> :synth ((∀ a : Type 1, Wrap a) → Wrap (∀ b : Type, b → b))
  it1  fun x => x _
λ> :set synth-engine exference
synth engine: exference
λ> :synth (∀ (F : Type 1 → Type), (∀ a : Type 1, F a) → F (∀ b : Type, b → b))
  it1  fun _ x => x _
```

The serializer admits only first-order kind arrows between universes and only application arguments whose own type is a universe. Term-indexed families such as `P 3` remain opaque. Unsaturated structural built-ins `And`, `Prod`, `PProd`, `Or`, `Sum`, `PSum`, `Iff`, and `Not` remain excluded as higher-kinded assignment heads until their unsaturated renderer identity is modeled; saturated uses keep their structural translation. A retained nominal application can support a positive, Lean-verified candidate but always poisons a negative verdict because its constant hides structure. Query-derived candidate types come only from proven proper-type positions below arrows and tuples in the sequent, including quantified subtrees already present there; provider-local assignment arguments may also come from the bounded active-instance channel above. This is guarded, evidence-directed impredicativity, not general second-order search. The query-derived visible-provider route is capped at four binders and 32

argument combinations; Djex's internal Haskell-instance route remains monotype-only. The external Lean evidence route has the separate four-binder, 32-head, 16-vector-per-provider, and 32-vector limits. Open, incomplete, wrong-arity, or context-bearing vectors fail closed, and chains with five or more leading binders remain a deliberate incomplete boundary. The original local-provider contract, scalar active-instance extension, and correlated follow-up are recorded in the scoped local-provider report and the provider-local instance-head report, followed by the correlated assignment report.

Unit regressions pin the kinded wire format, kind and argument order, finite bounds, whole-vector deduplication, and the exact vacuous applications under Djinn, Exference, and combined mode. The live `synth-provider-higher-kind-assignment` transcript retains the mixed higher-kinded/rank-N vector, the heterogeneous arity-one/arity-two multi-vacuous vector, and both exact `Wrap` and `Pair Nat` alternatives for one provider. Engine-specific ordering remains ranking policy.

5.4 Impossibility, proved

When Djinn's complete search exhausts a fully translated goal, failure is a theorem:

session

```
λ> :synth (∀ a b : Type, a → b)
provably uninhabited — no closed term of this polymorphic type exists
```

No Lean tactic gives you this answer (a failing `exact?` proves nothing). Note the careful wording: the verdict is about *closed terms of the polymorphic type*. It does not say any particular instance $a \to b$ is empty (choose $a = b$ and it isn't), and for propositions it is about *constructive* provability, a distinction §5.7 exploits. When the translation had to hide structure behind an opaque atom, the verdict backs off to match:

session

```
λ> :synth (∀ a : Type, Sigma (fun _ : Nat => a))
no term found within bounds (opaque atoms `Nat` hide structure the engine cannot
↪  analyze — not a refutation)
```

5.5 Inductive types

A non-recursive, non-indexed inductive or structure applied to all its parameters is expanded into a generalized sum of products: constructors become introduction rules, case analysis the elimination rule. This covers the standard library's little workhorses —

session

```
λ> :synth Ordering
  it1  .lt
  it2  .eq
  it3  .gt
λ> :synth (∀ a : Type, Option (Option a) → Option a)
  it1  fun _ _ => .none
```

```
  it2  fun _ x => match x with | .none => (.none) | .some y => y
λ> :synth (∀ e a b : Type, Except e a → (a → b) → Except e b)
  it1  fun _ _ _ x f => match x with | .error y => .error y | .ok z => .ok (f z)
```

— `Option.join` and `Except.map` synthesized rather than remembered. It extends to `Type`-valued classes-as-data like `Decidable`, where the eliminations write themselves, up to and including the instance combinators a library author would otherwise write by hand:

session

```
λ> :synth (∀ p : Prop, Decidable p → Decidable (¬ p))
  it1  fun _ x => match x with | .isFalse k => .isTrue k | .isTrue y => .isFalse (fun
  ↪  k1 => k1 y)
λ> :synth (∀ p q : Prop, Decidable p → Decidable q → Decidable (p ∧ q))
  it1  fun _ _ x y => match x with | .isFalse k => .isFalse (fun ⟨z, _⟩ => k z) |
  ↪  .isTrue w => (match y with | .isFalse k1 => .isFalse (fun ⟨_, x1⟩ => k1 x1) |
  ↪  .isTrue x2 => .isTrue ⟨w, x2⟩)
λ> :synth (∀ p q : Prop, Decidable p → Decidable q → Decidable (p → q))
  it1  fun _ _ x y => match x with | .isFalse k => .isTrue (fun z => absurd z k) |
  ↪  .isTrue w => (match y with | .isFalse k1 => .isFalse (fun f => k1 (f w)) |
  ↪  .isTrue x1 => .isTrue (fun _ => x1))
```

The last two are decidability of conjunction and of implication, assembled by case analysis on both instance arguments: four branches each, every branch carrying its own little proof.

Session-declared types participate the moment you declare them, and refutations over expanded inductives remain sound, because the engine saw the complete constructor list:

session

```
λ> structure Pair (A B : Type) where
…>   fst : A
…>   snd : B
…>
λ> :synth (∀ a b : Type, a → b → Pair a b)
  it1  fun _ _ x y => ⟨x, y⟩
λ> :synth (∀ a b : Type, Pair a b → Empty)
provably uninhabited — no closed term of this polymorphic type exists
```

Recursive types cannot be expanded; elimination is where undecidability lives. Their *constructors* are still sound introduction rules, though, so building is possible even where analysis is not:

session

```
λ> :synth Nat
  it1  Nat.zero
  it2  Nat.succ Nat.zero
λ> :synth (∀ a : Type, a → List a)
  it1  fun _ _ => List.nil
  it2  fun _ x => List.cons x List.nil
```

5.6 Recursion from the library

`:synth` will never invent a `Nat.rec`-based program, but for the everyday recursive types it does not have to: the library already wrote the recursion. A goal that mentions `List` or `Nat` brings a rated inventory of library functions with it — `List.map`, `List.foldr`, `List.append`, `List.flatten`, `List.length`, `List.replicate`, `Nat.add`, … — instantiated at the goal's own types and offered to the engine as extra premises. The enumeration prefers proofs that use the goal's own arguments, generally putting a direct library answer first while retaining distinct choices between same-typed arguments:

session

```
λ> :synth ((a → b) → List a → List b)
  it1  fun f x => List.map f x
  it2  fun f x => List.reverse (List.map f x)
λ> :synth (List (List a) → List a)
  it1  fun x => List.flatten x
  it2  fun x => List.reverse (List.flatten x)
λ> :synth (Nat → a → List a)
  it1  fun x y => List.replicate x y
  it2  fun x y => List.reverse (List.replicate x y)
λ> :synth (List a → List b → List (a × b))
  it1  fun x y => List.zip x y
  it2  fun _ _ => List.nil
λ> :synth ((a → b → c) → List a → List b → List c)
  it1  fun f x y => List.zipWith f x y
  it2  fun _ _ _ => List.nil
λ> :synth (List a → List a → List a)
  it1  fun x _ => x
  it2  fun _ x => x
  it3  fun x _ => List.reverse x
  it4  fun x y => List.append x y
  it5  fun x y => List.append y x
```

(Trailing candidates and the truncation notes are elided here.) The inventory is a ratings list in Djex's `*.ratings` format — lower is better, 100 or more disables — and a project file `leant.ratings` (lines of `Name Rating`, `#` comments) merges over the defaults at startup, so re-ranking, disabling, or growing the inventory is editing a list, not writing code.

The library search runs beside the constructor search of the previous section, in a mode where the recursive occurrences are sealed atoms: without `nil`/`cons` in play, every candidate must route through the goal's own arguments and the offered functions, which is what keeps `List.map` ahead of the endless supply of closed terms that ignore the input. Both searches' survivors are shown together, library candidates first; `:set synth-library off` restores the constructors-only behavior. As always, the premises are only *offers*: every candidate is re-elaborated by the backend, so a useless premise costs search time, never a wrong answer, and negative verdicts never come from the library run at all.

5.7 Classical candidates

Constructive logic goes further than newcomers expect, and `:synth` stays inside it whenever it can. Triple negation collapses, and contraposition holds in one direction, with no axioms in sight:

session

```
λ> :synth (∀ p : Prop, ¬¬¬p → ¬p)
  it1  fun _ k x => k (fun k1 => k1 x)
λ> :synth (∀ p q : Prop, (p → q) → ¬q → ¬p)
  it1  fun _ _ f k x => k (f x)
```

Peirce's law is different: it has no constructive inhabitant, and `:synth` can prove that. But rather than stopping at the refutation, it first gives constructive live providers a chance to supply a Lean-verified term. Only if those lanes fail does it ask the classical question, and answer with a term whose spelling shows what was used:

session

```
λ> :synth (∀ p q : Prop, ((p → q) → p) → p)
  it1  fun _ _ f => match Classical.em _ with | .inl x => x | .inr k => f (fun y =>
  ↪  absurd y k)
λ> :synth (∀ p : Prop, ¬¬p → p)
  it1  fun _ k => match Classical.em _ with | .inl x => x | .inr k1 => absurd k1 k
```

These are the proofs a human would write: one case split on excluded middle, one `absurd` where the contradiction lands. The hard direction of De Morgan needs excluded middle twice, once per disjunct, and the candidate reads as exactly that case analysis:

session

```
λ> :synth (∀ p q : Prop, ¬(¬p ∧ ¬q) → p ∨ q)
  it1  fun _ _ k => match Classical.em _ with | .inl x => .inl x | .inr k1 => (match
  ↪  Classical.em _ with | .inl y => .inr y | .inr k2 => absurd ⟨k1, k2⟩ k)
```

Less famous gaps between the two logics come out the same way. Weak excluded middle, and the Gödel–Dummett formula:

session

```
λ> :synth (∀ p : Prop, ¬p ∨ ¬¬p)
  it1  fun _ => match Classical.em _ with | .inl x => .inr (fun k => k x) | .inr k1 =>
  ↪  .inl k1
λ> :synth (∀ p q : Prop, (p → q) ∨ (q → p))
  it1  fun _ _ => match Classical.em _ with | .inl _ => (match Classical.em _ with |
  ↪  .inl x => .inl (fun _ => x) | .inr k => .inr (fun y => absurd y k)) | .inr k1 =>
  ↪  .inl (fun z => absurd z k1)
```

Two searches stand behind this. First, one excluded-middle assumption per atomic subformula is handed to the intuitionistic engine — complete for the propositional fragment, since intuitionistic logic plus atom instances of $p \vee \neg p$ *is* classical propositional logic. The findings are rendered as `Classical.em` case splits. If that misses, the goal's double-negation

translation runs instead (complete by Glivenko's theorem) and the candidate is wrapped in `Classical.byContradiction`. Both kinds are verified like any other candidate. The whole behavior sits behind a switch:

session

```
λ> :set synth-classical off
synth classical: off
λ> :synth (∀ p : Prop, p ∨ ¬ p)
provably uninhabited — no closed term of this polymorphic type exists
(constructively — a classical proof may still exist; this is not a disproof of the
↪  proposition)
```

5.8 Dependent formulas as cargo

Leant's translation makes no attempt to analyze dependent types. It does not refuse them either: a dependent subformula becomes an opaque *atom*, compared up to α-equivalence, which the engine can carry from one side of a goal to the other. Transport works; analysis does not:

session

```
λ> opaque P : Nat → Prop
λ> opaque Q : Prop
λ> :synth ((∀ n : Nat, P n) ∧ Q → Q ∧ (∀ n : Nat, P n))
  it1  fun ⟨x, y⟩ => ⟨y, x⟩
```

The engine never looked inside $\forall n,\ Pn$; it swapped a sealed box. A goal that would require opening the box (an induction, a rewrite, a case split on an index) is refused with a reason; that work belongs to `:prove`.

5.9 Synthesis inside a proof

Bare `:synth` in prove mode targets the current goal *with its hypotheses as premises.* Candidates are offered as `it1`, `it2`, … exactly as in the session, except that here `itN` splices the candidate applied to your hypotheses, so `exact it1` closes the goal. The hypotheses can carry real weight; below, the case analysis on the `Decidable` instance comes from `hd`:

session

```
λ> :prove ∀ p q : Prop, Decidable p → (p → q) → (¬p → q) → q
entering prove mode — type tactics; :help for commands
⊢ ∀ (p q : Prop) (a : Decidable p), (p → q) → (¬p → q) → q
suggestion: exact fun p q a a_1 a_2 => dite p a_1 a_2  (closes the goal)
⊢> intro p q hd hpq hnq
p q : Prop
hd : Decidable p
hpq : p → q
hnq : ¬p → q
⊢ q
suggestion: exact dite p hpq hnq  (closes the goal)
⊢> :synth
```

```
(synthesizing with hypotheses p q hd hpq hnq as premises)
  it1  fun _ _ x f g => match x with | .isFalse k => g k | .isTrue y => f y
⊢> exact it1
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed byCases
saved: theorem byCases : ∀ p q : Prop, Decidable p → (p → q) → (¬p → q) → q
```

Note the contrast with the tactic suggestion above it: the suggestion *found* the library's `dite`, while `:synth` *composed* the case split from the goal's own material. Both are verified; they simply answer different questions.

It also slots into an ordinary multi-goal proof. Split a conjunction with `constructor` and let `:synth` deal with each half in turn. Notice the prompt tracking the number of open goals, and the projections from `h` showing up without being asked for:

session

```
λ> :prove ∀ p q r : Prop, (p → q) ∧ (q → r) → p → r ∧ (p → q)
entering prove mode — type tactics; :help for commands
⊢ ∀ (p q r : Prop), (p → q) ∧ (q → r) → p → r ∧ (p → q)
suggestion: intro p q r h h1 <;> simp_all  (closes the goal)
⊢> intro p q r h hp
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ r ∧ (p → q)
suggestion: simp_all only [forall_const, imp_self, and_self]  (closes the goal)
⊢> constructor
— goal 1 of 2 —
case left
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ r
— goal 2 of 2 —
case right
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ p → q
suggestion: simp_all only [forall_const]  (closes the goal)
⊢2> :synth
(synthesizing with hypotheses p q r h hp as premises)
  it1  fun _ _ _ ⟨f, g⟩ x => g (f x)
⊢2> exact it1
case right
p q r : Prop
h : (p → q) ∧ (q → r)
hp : p
⊢ p → q
suggestion: simp_all only [forall_const, imp_self]  (closes the goal)
⊢> :synth
(synthesizing with hypotheses p q r h hp as premises)
```

```
  it1  fun _ _ _ ⟨f, _⟩ x _ => f x
⊢> exact it1
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed chain
saved: theorem chain : ∀ p q r : Prop, (p → q) ∧ (q → r) → p → r ∧ (p → q)
```

Unlike `exact?`, which finds an *existing* lemma, this composes a new term from the goal's own material: a constructive complement to the finisher tactics, needing no premise database and no imports. The division of labor that emerges is pleasant: the genuinely dependent moves (quantifiers, induction, rewriting) stay yours, and the propositional bookkeeping between them does not.

The classical fallback of §5.7 follows `:synth` into prove mode. Double-negation elimination has no constructive proof, so the candidate arrives as an excluded-middle case split, and `:qed` turns it into a named theorem — no tactic beyond `exact` was typed:

session

```
λ> :prove ∀ p : Prop, ¬¬p → p
entering prove mode — type tactics; :help for commands
⊢ ∀ (p : Prop), ¬¬p → p
suggestion: exact fun p a => Classical.byContradiction a  (closes the goal)
⊢> :synth
  it1  fun _ k => match Classical.em _ with | .inl x => x | .inr k1 => absurd k1 k
⊢> exact it1
All goals accomplished
finish with :qed [NAME], inspect with :script
⊢> :qed not_not_elim
saved: theorem not_not_elim : ∀ p : Prop, ¬¬p → p
```

5.10 Engines and budgets

The default engine is Djinn: complete, terminating, and the only source of refutations. `:set synth-engine exference` switches to the ranked heuristic search, and `both` runs the two together — negative verdicts still come only from Djinn. Standalone lanes send at most 12 fresh candidate groups to Lean. A combined lane gets 24 and retains both standalone frontiers: writing D and E for fresh Djinn and Exference groups, its order is `D1–D4, E1–E12, D5–D12`, then alternating tails. Exference runs under explicit budgets (`:set synth-steps`, default 4096), and truncation is always reported:

session

```
λ> :set synth-engine both
synth engine: both
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1  fun f g x => f x (g x)
note: exference: search truncated: step limit reached, queue limit pruned 31990
```

Every engine mode first gives a structurally accepted goal a provider-free lane and asks Lean to verify its candidates. When no provider-free term verifies, Lean discovers at most 80 live term providers under the namespaces used by the target (plus exact session declarations). A complete Djinn refutation is retained as a sound fallback while those constructive provider lanes run: the first candidate accepted by Lean wins; an empty, unavailable, timed-out, or unsuccessful provider search restores the refutation before any classical retry. The focused `synth-provider-refutation-fallback` transcript checks exact rank-N overrides under Djinn, Exference, and combined mode. It also enables classical search for a Peirce-shaped goal and confirms that an exact constructive provider wins first, while a no-provider control retains the original sound refutation. Pure tests pin the same direct, rank-N, and unusable-provider boundaries below the REPL layer. Type constructors and type families, whose fully peeled result is a sort, are excluded before search. Exact-result session and public declarations lead the ranking; conventional implementation-worker names remain eligible but follow public fallbacks, while session declarations bypass that naming heuristic. Exference searches the ranked inventory directly; Djinn tries discovery-order prefixes of 1, 4, and 16 providers before the full bounded inventory, and may specialize a loaded scheme at a closed monotype or guarded rank-N polytype. The serializer derives a canonical query from the sorted, deduplicated namespace roots and final result head, rather than from the goal's raw spelling. Successful inventories — including an empty one — are kept in a generation-aware 12-entry LRU. Imports, ordinary declarations, and session-restoring operations invalidate that generation; generated `it1`, `it2`, … bindings cannot be providers and preserve it.

For an exact provider with erased instance constraints, the same discovery pass can attach bounded active-instance-head assignments. Resolution follows Lean's own instance order and isolates the selected head plus its complete constraint closure. One successful head yields one ordered vector; an incomplete or unusable head yields nothing. Provider slicing retains the vector only with its source declaration; after whole-vector context/depth and arity filtering Leant takes the command-wide prefix before planning or translation. Each argument crosses with its bounded `Type`-arrow kind; Leant reconstructs a Djex `GroundKind` and calls the checked kinded Djinn or Exference assignment runner. Leant rejects residual kind arities above 64 before that bridge, and pinned Djex independently rejects a supplied `GroundKind` above 129 constructor nodes before recursive operations on that assignment. Lean still reconstructs every dictionary during final elaboration—only the closed kind/type vector crosses the bridge.

Exact rendered spellings keep their first scheduled occurrence. Before a later provider lane is forced or capped, spellings already rejected by Lean are removed from each engine's source stream and newly empty groups are dropped. Rediscovered failures therefore spend none of that lane's 12- or 24-group fresh frontier.

The pure searches answer in microseconds to milliseconds; the cost center is backend verification, a few hundred milliseconds per candidate. A wall-clock guard (default 20 s, `LEANT_SYNTH_TIMEOUT`) covers the quantified goals whose bounded instantiation can widen the space; hitting it is reported as "no answer," never as a verdict.

5.11 What `:synth` will not do

Some plain statements of present limits. It will not synthesize recursion or induction. Complete compatible recursive families contribute bounded positive construction to Djinn and one constructor layer of elimination to Exference; recursive fields are never opened again. It will not analyze dependent structure; atoms are transported, never opened. Opaque and bound-variable proper-type applications retain their ordered arguments, and qualifying inductives share one exact parametric family across distinct `Option`/`List` occurrences. Term-indexed, dependent, ambiguous, or incompatible families keep their conservative fallback. Quantifier handling beyond the propositional fragment follows Djex's bounded rank-N and guarded impredicativity rules, which are sound and terminating over an undecidable space and widen as Djex improves. The current plan family is exhaustive through nine independent quantified sites; a ten-site five-open/five-opaque goal is the next deliberate occurrence-plan gap, while hypothesis instantiation chains with five or more binders remain opaque. Such higher-rank goals answer "no term found within bounds" and nothing stronger. And although Djex can resolve type-class evidence itself, Leant leaves instance arguments to Lean's elaborator during verification, which sits nearer the instance tables anyway. A non-dependent instance-implicit goal binder is retained as a render-only slot: it contributes no dictionary premise to either engine, but Leant inserts its wildcard before later lambda arguments and withholds complete negative evidence across the erased boundary. Constrained rank-N hypotheses therefore keep their instance applications implicit without shifting the candidate's term binders. Provider discovery now consults active Lean instance heads only to obtain bounded ground-kinded assignments for the exact provider whose erased constraints they collectively solve. Constraint-only vacuous higher-kinded binders are supported when each argument kind is a bounded `Type -> … -> Type` chain; unsaturated structural built-in heads remain excluded. Leant never serializes dictionaries or general instance resolution into Djex. A selected head is accepted only after its own subgoals and every remaining provider constraint close in one isolated metavariable context. Up to 32 heads are inspected and 16 distinct complete vectors retained per provider, with at most 32 vectors passed in total; open, incomplete, wrong-arity, depth-truncated, and contextual vectors fail closed. The reducible control `Gap.Contextual := {a : Type} → [Inhabited a] → a` is therefore excluded even when an active class instance names it. This provider-local channel can expose a polytype absent from the query, but it is bounded checked evidence rather than general impredicative inference. The fragment is bounded on purpose: inside it, answers are fast and verified. Refutations are complete for the provider-free structural calculus but provisional with respect to the live environment. Leant searches the bounded, best-effort provider inventory before reporting one: a verified provider candidate overrides it, while an empty or unsuccessful provider search preserves it as the original sound fallback. The resulting verdict is not an exhaustive theorem about every declaration or axiom in the Lean environment.

6 How it works, briefly

A `:synth` query passes through five checked stages. A Lean metaprogram (compiled once into a cached side environment) elaborates the goal and serializes it into the engines' fragment: structural connectives are decomposed; qualifying inductives expand into constructor lists;

proper-type applications retain rigid or bound heads and ordered arguments; everything else becomes an α-normalized opaque atom. The fragment translator either accepts the result or refuses with a reason. The cached synthesis environment remembers the session history it has replayed: an unchanged history is reused, an append replays only its new suffix, and undo or another non-prefix change rebuilds from the cached import-and-serializer base. Generated result bindings participate in this replay so later goals can mention them without flushing the provider cache. The selected engine searches; its candidates are rendered back into Lean syntax, with constructor names restored, binders named by role, and anonymous-constructor and leading-dot spellings preferred over fully qualified fallbacks. Finally the backend re-elaborates each candidate against the original goal, and only survivors are shown and bound. The engine boundary is narrow by design: the searches behind it are swappable, and nothing they emit is believed until Lean has type-checked it.

An unpickled environment is an explicit history barrier. Leant copies it to a private process-lifetime artifact so backend restart can restore that base before replaying the commands entered afterward; the reconstructed undo stack therefore still bottoms out at the snapshot. `:reset` and `:load` leave snapshot mode, whereas `:import` requires that explicit transition because imports cannot be added to an opaque Lean environment. A Leant `:pickle` also writes a content-fingerprinted, serializer-ABI-keyed synthesis companion. Foreign or stale sidecars are ignored without rejecting the main upstream snapshot; if that snapshot exposes Lean's metaprogramming API, Leant compiles current synthesis tooling directly over it.

7 Status

Leant is a working prototype under active development. The synthesis design and its phased history are documented in `docs/SYNTHESIS_PROPOSAL.md`; transcript-level golden tests live in `test/`. The project began inside the *ProveIt* repository and was split out with its history intact. Expect sharp edges: the project exists to find out how far a conversational, verification-gated synthesis workflow can be pushed, and it moves at the speed of that question.